MIS780 Advanced AI For Business - Assessment 2 - T2 2025

Task 1: Solar power optimization

Student Name: Phuong Viet Dang

Student ID: 224383373

Table of Content

1. Executive Summary
2. Data Preprocessing
3. Predictive Modeling
4. Experiments Report
5. Role of GenAI

1. Executive Summary

The business problem addressed in this task is the optimisation of solar farm operations through accurate prediction of solar power generation. The dataset contains 21 meteorological and solar geometry variables (e.g., temperature, humidity, cloud cover, wind, angle of incidence, zenith, azimuth) and one target variable, generated_power_kw.

Data preprocessing involved removal of the index column, handling missing values via median imputation, and MinMax scaling of features. Four models were developed and compared: a Linear Regression baseline and three Multilayer Perceptron (MLP) models with increasing complexity.

Experimental results showed that the deeper MLP (128–64–32 with dropout) outperformed the baseline, achieving the best performance with RMSE = 442.2, MAE = 296.6, and R^2 = 0.77, compared with Linear Regression (RMSE = 503.5, R^2 = 0.70). This confirms that solar power output depends on non-linear interactions between predictors. The recommended solution is the three-layer MLP model, which can provide reliable forecasts to support grid scheduling, storage management, and operational decision-making in solar farms.

2. Data Preprocessing

1. Cleaning: Dropped the Index column as it carried no predictive information.
2. Missing values: There is no missing value detected.
3. Scaling: MinMaxScaler was used to transform features into the [0,1] range. Validation values exceeded [0,1] slightly due to unseen extremes, which is expected and aligns with lecture notes.
4. Split: The dataset was divided into 70% training and 30% validation sets.
5. Conversion: Features and labels were converted into NumPy arrays for compatibility with scikit-learn and TensorFlow/Keras model
6. Using Linear regression as a baseline KPI for evaluating. RMSE: 503.5056121514164 MAE: 387.12912230762225 R^2: 0.7047321533730386

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

```
SEED = 2020
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

In [ ]:
```
df = pd.read_csv("Task 1 Dataset Solar Power.csv")

# Drop index column
if "Index" in df.columns:
    df.drop(columns=["Index"], inplace=True)

label_col = "generated_power_kw"
print("Shape:", df.shape)
df.head()
```

```
Shape: (4213, 21)
```

Out[ ]:

	temperature_2_m_above_gnd	relative_humidity_2_m_above_gnd	mean_sea_level_pressure_MSL	total_precipitation_sfc	snowfall_a
0	2.17	31	1035.0	0.0	
1	2.31	27	1035.1	0.0	
2	3.65	33	1035.4	0.0	
3	5.82	30	1035.4	0.0	
4	7.73	27	1034.4	0.0	

5 rows × 21 columns

In [ ]:
```
print("Dtypes:\n", df.dtypes)

numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = [c for c in df.columns if c not in numeric_cols]

print("Numeric columns:", len(numeric_cols))
print("Categorical columns:", categorical_cols or "None")
```

```
Dtypes:
 temperature_2_m_above_gnd            float64
relative_humidity_2_m_above_gnd        int64
mean_sea_level_pressure_MSL          float64
total_precipitation_sfc              float64
snowfall_amount_sfc                  float64
total_cloud_cover_sfc                float64
high_cloud_cover_high_cld_lay          int64
medium_cloud_cover_mid_cld_lay         int64
low_cloud_cover_low_cld_lay            int64
shortwave_radiation_backwards_sfc    float64
wind_speed_10_m_above_gnd            float64
wind_direction_10_m_above_gnd        float64
wind_speed_80_m_above_gnd            float64
wind_direction_80_m_above_gnd        float64
wind_speed_900_mb                    float64
wind_direction_900_mb                float64
wind_gust_10_m_above_gnd             float64
angle_of_incidence                   float64
zenith                               float64
azimuth                              float64
generated_power_kw                   float64
dtype: object
Numeric columns: 21
Categorical columns: None
```

In [ ]:
```
train_df, test_df = train_test_split(df, train_size=0.7, test_size=0.3, random_state=SEED)

X_train = train_df.drop(columns=[label_col])
y_train = train_df[[label_col]]
X_test = test_df.drop(columns=[label_col])
y_test = test_df[[label_col]]

print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
```

```
Training samples: 2949
Test samples: 1264
```

In [ ]:
```
print("Missing values before imputation (train):", X_train.isnull().sum().sum())
print("Missing values before imputation (test):", X_test.isnull().sum().sum())
```

```
Missing values before imputation (train): 0
Missing values before imputation (test): 0
```

In [ ]:
```
imputer = SimpleImputer(strategy="median")
X_train = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.columns, index=X_train.index)
```

```
X_test = pd.DataFrame(imputer.transform(X_test), columns=X_test.columns, index=X_test.index)

print("After imputation (train):", X_train.isnull().sum().sum())
print("After imputation (test):", X_test.isnull().sum().sum())
```

```
After imputation (train): 0
After imputation (test): 0
```

In [ ]:
```
scaler = MinMaxScaler()
X_train = pd.DataFrame(scaler.fit_transform(X_train), columns=X_train.columns, index=X_train.index)
X_test = pd.DataFrame(scaler.transform(X_test), columns=X_test.columns, index=X_test.index)

print("X_train range:", (X_train.min().min(), X_train.max().max()))
print("X_test range:", (X_test.min().min(), X_test.max().max()))
```

```
X_train range: (0.0, 1.0000000000000002)
X_test range: (-0.0059259259259259265, 1.0056390977443608)
```

In [ ]:
```
arr_x_train  = X_train.values
arr_y_train  = y_train.values
arr_x_test  = X_test.values
arr_y_test  = y_test.values
print("arr_x_train:", arr_x_train.shape)
print("arr_x_test:", arr_x_test.shape)
```

```
arr_x_train: (2949, 20)
arr_x_test: (1264, 20)
```

In [ ]:
```
lr = LinearRegression()
lr.fit(arr_x_train, arr_y_train)
y_pred_lr = lr.predict(arr_x_test)

rmse_lr = np.sqrt(mean_squared_error(arr_y_test, y_pred_lr))
mae_lr = mean_absolute_error(arr_y_test, y_pred_lr)
r2_lr = r2_score(arr_y_test, y_pred_lr)

print("Linear Regression — RMSE:", rmse_lr, "MAE:", mae_lr, "R²:", r2_lr)
```

```
Linear Regression — RMSE: 503.5056121514164 MAE: 387.12912230762225 R²: 0.7047321533730386
```

3. Predictive Modeling

Four models were implemented:

1. Linear Regression: Used as a baseline model to establish benchmark performance under linear assumptions.
2. MLP (64): One hidden layer with 64 neurons.
3. MLP (128–64): Two hidden layers with 128 and 64 neurons.
4. MLP (128–64–32 + Dropout): Three hidden layers (128, 64, 32 neurons) with 0.1 dropout to reduce overfitting.

All MLP models used ReLU activation, Adam optimiser, MSE loss function, and EarlyStopping with patience=20. Models were trained for up to 200 epochs with batch size 64.

In [ ]:
```
# Final solution: MLP (128-64-32 + Dropout, Deep model)
early_stop = EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True) #prevent overfitting

input_dim = arr_x_train.shape[1]
final_model = Sequential([
    Dense(128, activation="relu", input_shape=(input_dim,)),
    Dense(64, activation="relu"),
    Dense(32, activation="relu"),
    Dropout(0.1),
    Dense(1)
])
final_model.compile(optimizer="adam", loss="mse", metrics=["mae"])

history = final_model.fit(
    arr_x_train, arr_y_train,
    validation_data=(arr_x_test, arr_y_test),
    epochs=200, batch_size=64, verbose=0, callbacks=[early_stop]
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_sh
ape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

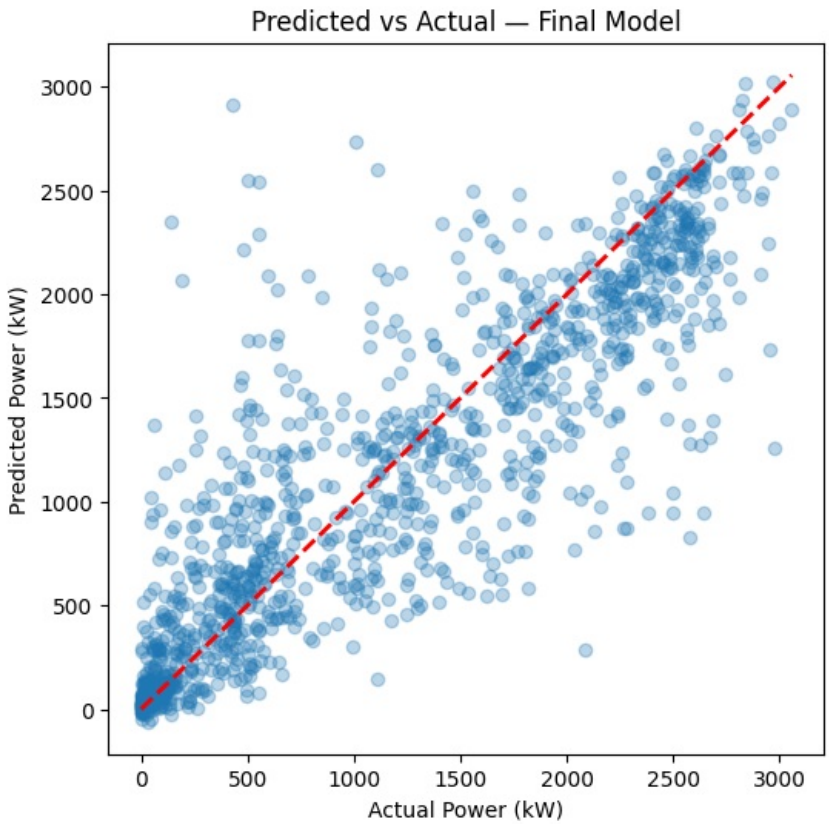
In [ ]:
```
# Evaluate on test set
y_pred_final = final_model.predict(arr_x_test, verbose=0)
rmse_f = np.sqrt(mean_squared_error(arr_y_test, y_pred_final))
```

```
mae_f  = mean_absolute_error(arr_y_test, y_pred_final)
r2_f   = r2_score(arr_y_test, y_pred_final)

print("Final model result:", "RMSE=", round(rmse_f,4), "| MAE=", round(mae_f,4), "| R²=", round(r2_f,4))

# Visual: Predicted vs Actual
plt.figure(figsize=(6,6))
plt.scatter(arr_y_test, y_pred_final, alpha=0.3)
lim_low, lim_high = float(arr_y_test.min()), float(arr_y_test.max())
plt.plot([lim_low, lim_high], [lim_low, lim_high], 'r--', lw=2)
plt.xlabel("Actual Power (kW)")
plt.ylabel("Predicted Power (kW)")
plt.title("Predicted vs Actual — Final Model")
plt.show()
```

```
Final model result: RMSE= 447.5821 | MAE= 302.6587 | R²= 0.7667
```

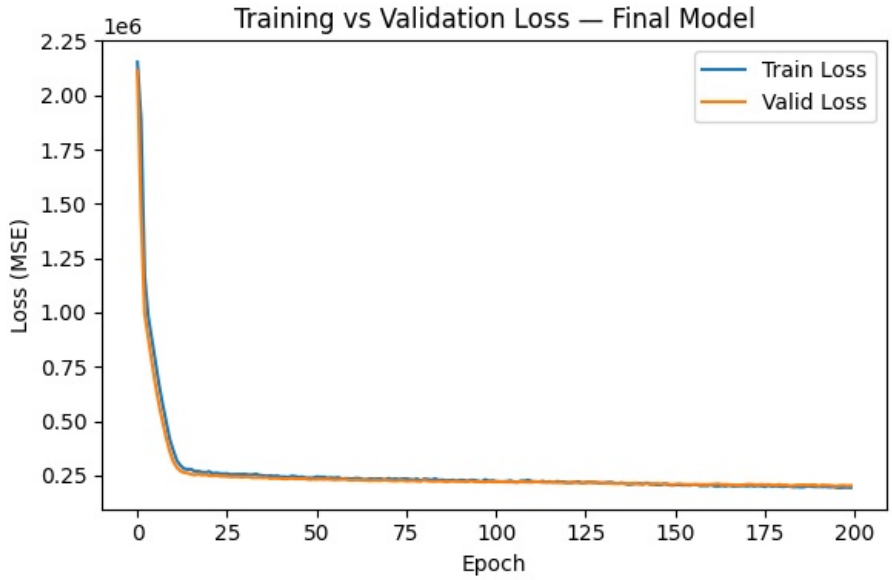


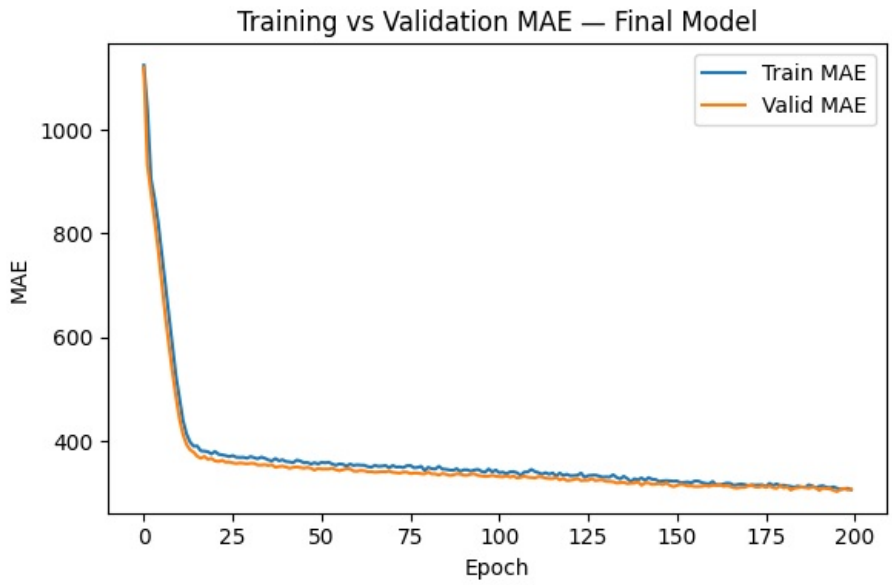
In [ ]:

```
# Plot training vs validation curves for the final model only

plt.figure(figsize=(6,4))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Valid Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Training vs Validation Loss — Final Model")
plt.legend(loc="upper right")
plt.tight_layout()
plt.show()

plt.figure(figsize=(6,4))
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Valid MAE')
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("Training vs Validation MAE — Final Model")
plt.legend(loc="upper right")
plt.tight_layout()
plt.show()
```

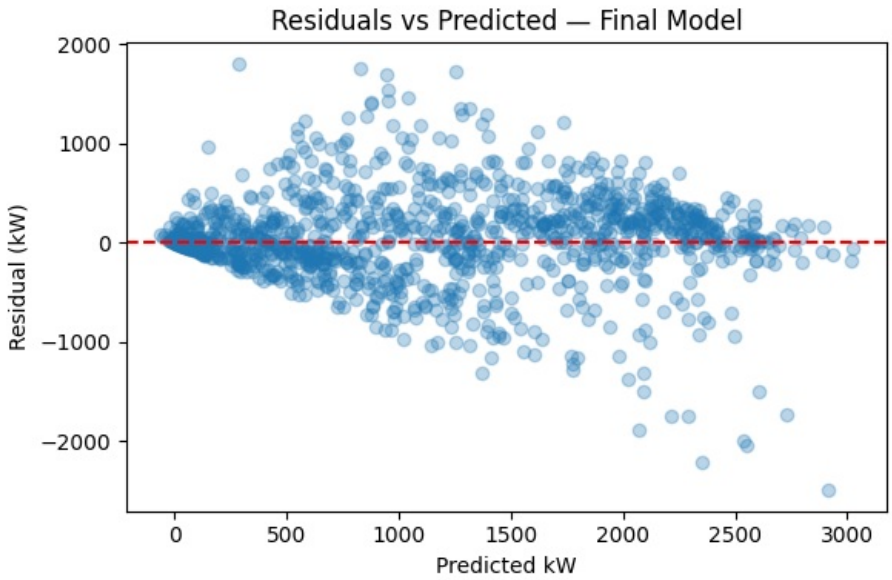


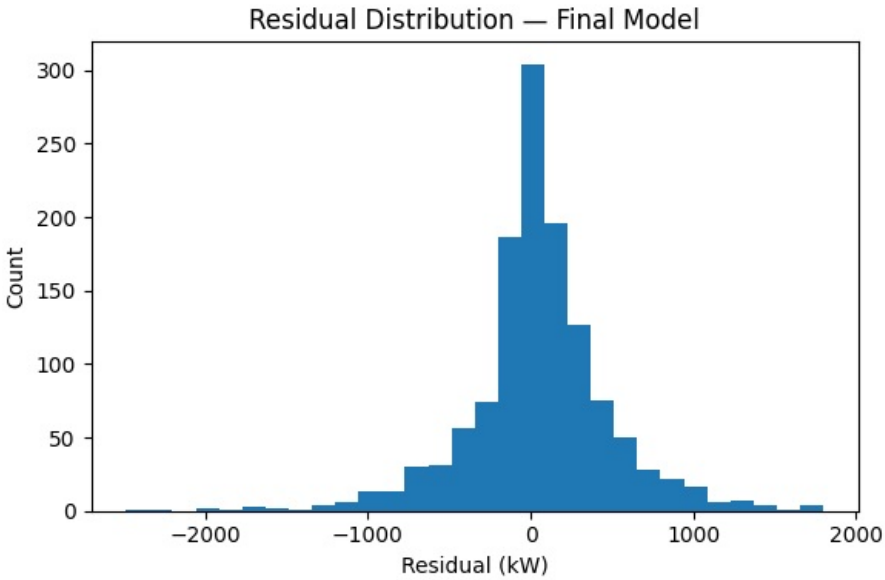


In [ ]:
```
# residuals on validation
y_pred_final = final_model.predict(arr_x_test, verbose=0)
residuals = (arr_y_test.flatten() - y_pred_final.flatten())

plt.figure(figsize=(6,4))
plt.scatter(y_pred_final, residuals, alpha=0.3)
plt.axhline(0, color='r', linestyle='--'); plt.xlabel("Predicted kW"); plt.ylabel("Residual (kW)")
plt.title("Residuals vs Predicted — Final Model"); plt.tight_layout(); plt.show()

plt.figure(figsize=(6,4))
plt.hist(residuals, bins=30); plt.title("Residual Distribution — Final Model")
plt.xlabel("Residual (kW)"); plt.ylabel("Count"); plt.tight_layout(); plt.show()
```





Explain the model

Final model building:

- This model framed solar power prediction with a supervised regression problem with a target variable of generated_power_kw.
- Arichitechture: the model is the deepest model out of the 4 since it using 3 hidden layers (128 - 64 - 32 + dropout). The dropout is a small value with only 0.1.
- RMSE, MAE, R^2 will be report as metrics for interpretability.
- Optimiser and training: using Adam, batch size = 64, epochs = 200.
- Data split 70% for training and 30% for testing.
- Pre-processing: median imputation, MinMax scaling.

Interpret the result

- RMSE = 456.7718, MAE = 318.5204, R^2= 0.757
- The training and validation curves stay close together across all epochs (loss & MAE plots), this indicate a good accuracy and generalisation with minimal overfitting.
- Residual diagnostics show errors centered around zero with wider spread at higher outputs (expected heteroscedasticity at peak generation). This supports the choice of a non-linear model and suggests possible future work on variance-aware loss or range-stratified tuning.
- The Predicted vs Actual scatterplot shows that the final MLP model captures the relationship between input variables and generated power with good accuracy. Most predictions align closely with the 45° line, indicating reliable forecasts. Errors are larger at higher power outputs, reflecting greater variability during peak solar generation periods. This suggests the model generalises well overall but could benefit from additional training data covering extreme weather/irradiance conditions.

Why this model was selected

- Multiple candidates has been evaluated (Linear Regression baseline and several MLP depths). The deeper MLP 128-64-32 + Dropout consistently delivered the best error and accuracy trade-off (lower RMSE / higher R^2) while remaining compact enough for practical deployment. Full experiment records and comparisons are included in the Experiment Report section; only this best model is presented here as the final solution.

How the model will be used

- The model ingests the latest weather/solar-geometry inputs and outputs a kW forecast to support solar-farm operations (generation scheduling, storage control, and grid alignment). It should be embedded behind a lightweight API, with periodic retraining as new seasons and operating conditions arrive.

4. Experiments Report

In [ ]:
```
# --- Helpers ---
def train_eval_mlp(layers, name, Xtr, ytr, Xte, yte, epochs=200, batch_size=64, dropout=None):
    model = Sequential()
    model.add(Dense(layers[0], activation="relu", input_shape=(Xtr.shape[1],)))
    for units in layers[1:]:
        model.add(Dense(units, activation="relu"))
    if dropout:
        model.add(Dropout(dropout))
    model.add(Dense(1))
```

```
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    model.fit(Xtr, ytr, validation_data=(Xte, yte),
              epochs=epochs, batch_size=batch_size, verbose=0,
              callbacks=[EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True)])
    y_pred = model.predict(Xte, verbose=0)
    return {
        "Model": name,
        "RMSE": np.sqrt(mean_squared_error(yte, y_pred)),
        "MAE":  mean_absolute_error(yte, y_pred),
        "R2":   r2_score(yte, y_pred),
        "Epochs_Run": epochs  # approximate; early stopping may stop earlier internally
    }
```

Baseline Linear Regression

In [ ]:
```
# --- Baseline: Linear Regression ---
lr = LinearRegression().fit(arr_x_train, arr_y_train)
y_pred_lr = lr.predict(arr_x_test)
res_lr = {
    "Model": "Linear Regression",
    "RMSE": np.sqrt(mean_squared_error(arr_y_test, y_pred_lr)),
    "MAE":  mean_absolute_error(arr_y_test, y_pred_lr),
    "R2":   r2_score(arr_y_test, y_pred_lr),
    "Epochs_Run": "-"
}
```

Experiments for multiple layer model

In [ ]:
```
# --- MLP experiments ---
res_mlp1 = train_eval_mlp([64], "MLP (64)", arr_x_train, arr_y_train, arr_x_test, arr_y_test)
res_mlp2 = train_eval_mlp([128, 64], "MLP (128-64)", arr_x_train, arr_y_train, arr_x_test, arr_y_test)
res_mlp3 = train_eval_mlp([128, 64, 32], "MLP (128-64-32 + DO0.1)", arr_x_train, arr_y_train, arr_x_test, arr_y_
```

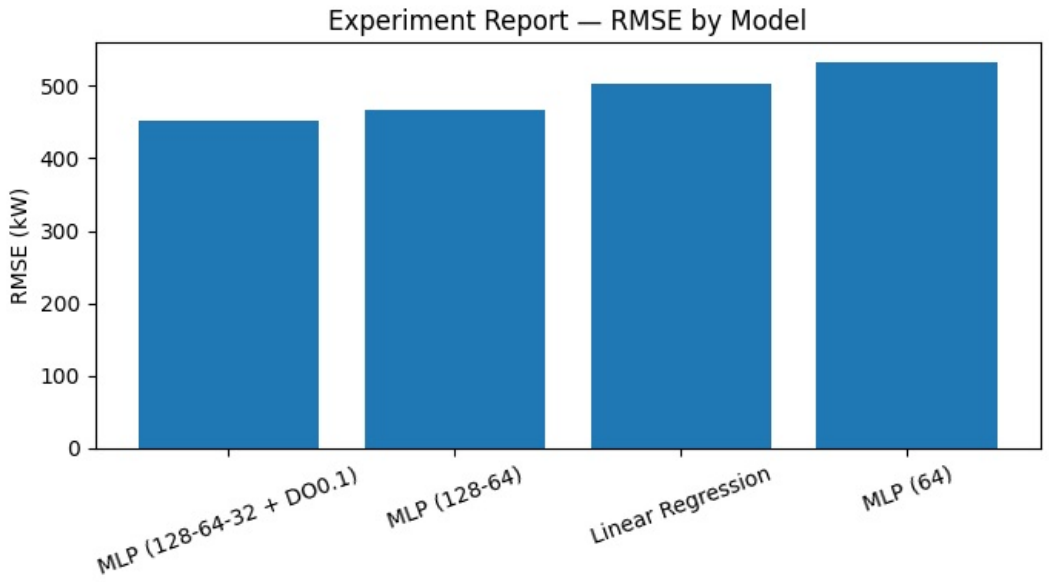
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_sh
ape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_sh
ape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_sh
ape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [ ]:
```
# --- Results table ---
df_results = pd.DataFrame([res_lr, res_mlp1, res_mlp2, res_mlp3])
df_results[["RMSE","MAE","R2"]] = df_results[["RMSE","MAE","R2"]].round(4)
df_results.sort_values("RMSE")
```

Out[ ]:

	Model	RMSE	MAE	R2	Epochs_Run
3	MLP (128-64-32 + DO0.1)	451.4828	307.0360	0.7626	200
2	MLP (128-64)	466.6481	328.6385	0.7464	200
0	Linear Regression	503.5056	387.1291	0.7047	-
1	MLP (64)	532.5405	422.1174	0.6697	200

In [ ]:
```
plt.figure(figsize=(7,4))
order = df_results.sort_values("RMSE")["Model"]
plt.bar(order, df_results.set_index("Model").loc[order, "RMSE"])
plt.ylabel("RMSE (kW)"); plt.title("Experiment Report — RMSE by Model")
plt.xticks(rotation=20); plt.tight_layout(); plt.show()
```



Comparing between multiple models

- As we could see from the table above, model 1 is too shallow and underfit which is the most inefficient model. Model 2 is deeper and improved in accuracy while model 3 is the best one with the deepest and best accuracy.
- Linear regression although explained around 70% of the variance but it also have high error, only second to model 1.
- Model 1: is the least efficent out of the 4 since only explained 66% of variance with highest error.
- Model 2: have some improve in accuracy compare to the others 2 with 73% of explained variance and second-lowest error.
- Model 3: explain around 75% of variance while have the lowest error. This one is the best model solution and has been named final_model in previous predictive modeling above.

Addressing the business problem

- The company need solar power to be predicted for solar power

1. The best time to operate the solar power generator to produce highest amount of power.
2. The best angle for the operators to use to optimize solar power.
3. Improving the efficieny of battery from reducing unecessary and low efficent cycle.
4. Reduce the cost by choosing the most efficient course and action with operators.

Deploy recommendation

- If we want to implement this model into real world, a few things must be considered.

1. Data quality: as we discuss, the model will be fluctuated in higher value which mean it will be require with reliable weather and sensors inputs. Also automated handling error and missing data must be in place.
2. Real-time intergration: Real time data to use for forcasting must be pulled directly from the real time table data.
3. Model retraining: Seasonal training and equipment aging must also be considered to retrain the model to achieve best accuracy in any given time.
4. Explainability: Build additional tools for operators to understand the model since deep learning could be hard to understand for non-tech customer.
5. Cost-benefits trade-off: The complex model will provide with better accuracy, however it will also consume more resource than simple model. Consider if the trade off is worth the complex or it should be fine with more casual model.

5. Role of GenAI

- I have used GenAI to improve my code and to see if my code need any fix or improvement, the prompt has been used: "Please access to this notebook file and let me know what should I improve with my code".

- ChatGPT was used to brainstorm ideas and list all of the steps must be done in each sections of the assignment.

- Chat Gpt was used to troubleshoot error code and improve help to improve the accuracy of the model.

- Chat GPT was used to write key ideas to brainstorm and list the work in a structured way.

While using GenAI: some obstacle has been meet with the over complicated while debugging and improve the code. I need to understand the code and remove part that does not used and teach in our lab session.